A GUIDED TOUR OF THE CODEBASE

How Fruit Wholesale Is Built

One SQL Server database, one ASP.NET Core API, and two clients — an Angular web app and a Flutter Android app — that both talk to it the same way. This is a tour of how each piece is put together, in the order data actually flows: the API first, since both clients are ultimately thin shells around it, then the Angular client, then the Android client.

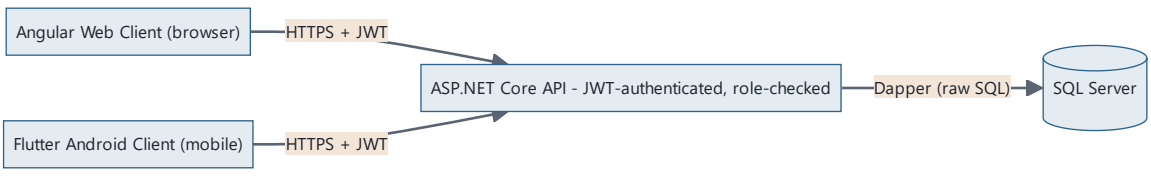
SQL Server — schema + stored procs

ASP.NET Core 10 — Clean Architecture API

Angular 20 — standalone components, signals

Flutter — Android client, Provider

CONTENTS

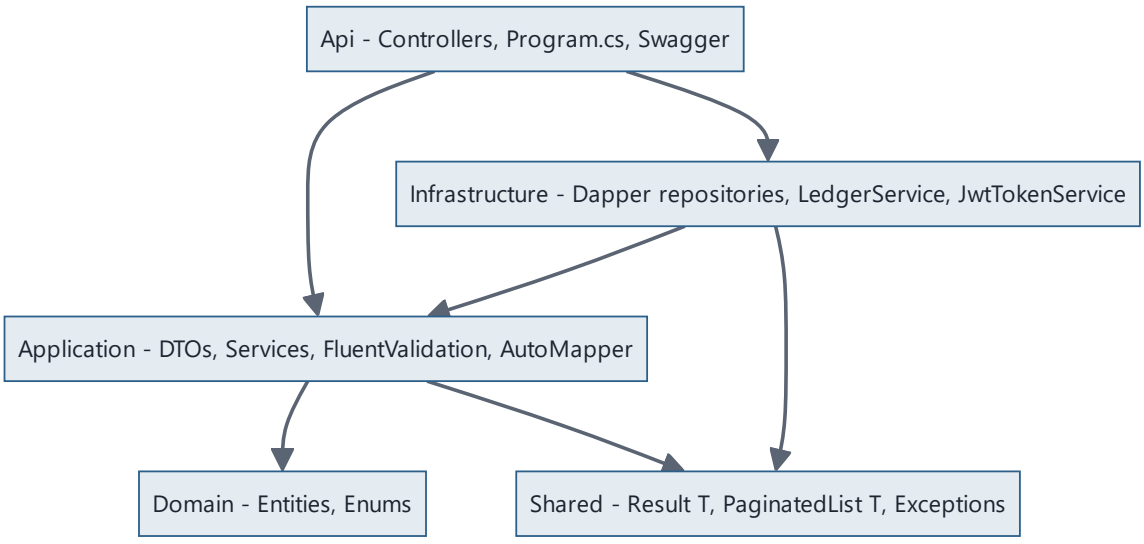


Both clients are interchangeable in the eyes of the API — same endpoints, same tokens, same role rules.

Every business rule — what counts as a valid invoice, who's allowed to see the Cash Ledger, how a shop's outstanding balance gets recalculated after a payment — lives exactly once, in the API. Neither client does anything the other can't; they're both just different-shaped forms and lists over the same 30-ish endpoints.

01 Backend API — Clean Architecture

Five projects, one dependency rule: **outer layers depend on inner layers, never the reverse.** Domain knows nothing about the database; the database access code knows nothing about HTTP; controllers know nothing about SQL.



Arrows mean "depends on." Domain and Shared have zero project references — they're the stable core everything else is built around.

Domain Shared Application Infrastructure Api

Domain Entities and enums, nothing else

Plain C# classes with properties — no attributes, no base classes, no dependency on Dapper or ASP.NET. `Collection.cs`, `Supply.cs`, `ShopMaster.cs` and friends live here, plus static classes for the system's controlled vocabularies:

src/FruitWholesale.Domain/Enums/DomainEnums.cs

```
public static class UserRoles
{
    public const string Admin = "Admin";
    public const string Manager = "Manager";
    public const string Accountant = "Accountant";
    public const string Staff = "Staff";
}
```

These four roles are the single source of truth for access control — the Angular guards, the Flutter nav filters, and the API's `[Authorize(Roles=…)]` attributes all ultimately point back to these same four strings.

Shared Result<T> and PaginatedList<T>

Two small types used everywhere. **Result<T>** replaces exceptions for expected business failures — "that username doesn't exist" isn't exceptional, it's a normal outcome a caller should handle without a stack trace:

src/FruitWholesale.Shared/Results/Result.cs

```
var result = await authService.LoginAsync(request);
if (!result.IsSuccess)
    return BadRequest(new { errors = result.Errors });
```

PaginatedList<T> is the uniform envelope every list endpoint returns — `items`, `pageNumber`, `pageSize`, `totalCount`, `totalPages`, `hasNextPage`, `hasPreviousPage`. Both clients parse the exact same shape, which is what let the Flutter app's list screens share one generic `PaginatedListView<T>` widget later on.

Application Where the business rules live

Every module — Supply, Purchase, Collection, SupplierPayment, DailyExpense, EmployeeWorkLog, and the six Masters — follows the same four-file shape:

DTOs
One file per module: read DTO, list-item DTO, create DTO, update DTO. Never the raw entity crosses the API boundary.

Services
Interface + implementation. Validates business rules, calls the repository, maps results, returns `Result<T>`.

Validators
FluentValidation classes wired into the pipeline — invalid requests never reach a service method.

AutoMapper profile
Entity ↔ DTO mapping declared once, reused everywhere that module's data moves.

Repository *interfaces* (`ISupplyRepository`, etc.) are declared here too, even though they're implemented one layer out in Infrastructure — that inversion is what keeps Application from depending on Dapper or SQL Server at all.

Infrastructure Dapper, not an ORM

No Entity Framework. Every repository is hand-written SQL through Dapper, which keeps the generated queries fully visible and predictable — important for a system where the numbers have to reconcile to the paisa.

src/FruitWholesale.Infrastructure/Repositories/CollectionRepository.cs

```
public async Task<Collection> CreateAsync(Collection collection)
{
    using var connection = new SqlConnection(_connectionString);
    await connection.OpenAsync();
    using var transaction = connection.BeginTransaction();

    // 1. insert the Collections row
    // 2. write the matching ShopLedger entry (via ILedgerService)
    // 3. EXEC sp_RecalculateShopBalance
    // 4. commit — all three succeed together, or none do
}
```

Api Controllers are thin on purpose

Every controller inherits `ApiControllerBase`, which carries the base `[Authorize]` attribute and one helper that turns a `Result<T>` into the right HTTP response:

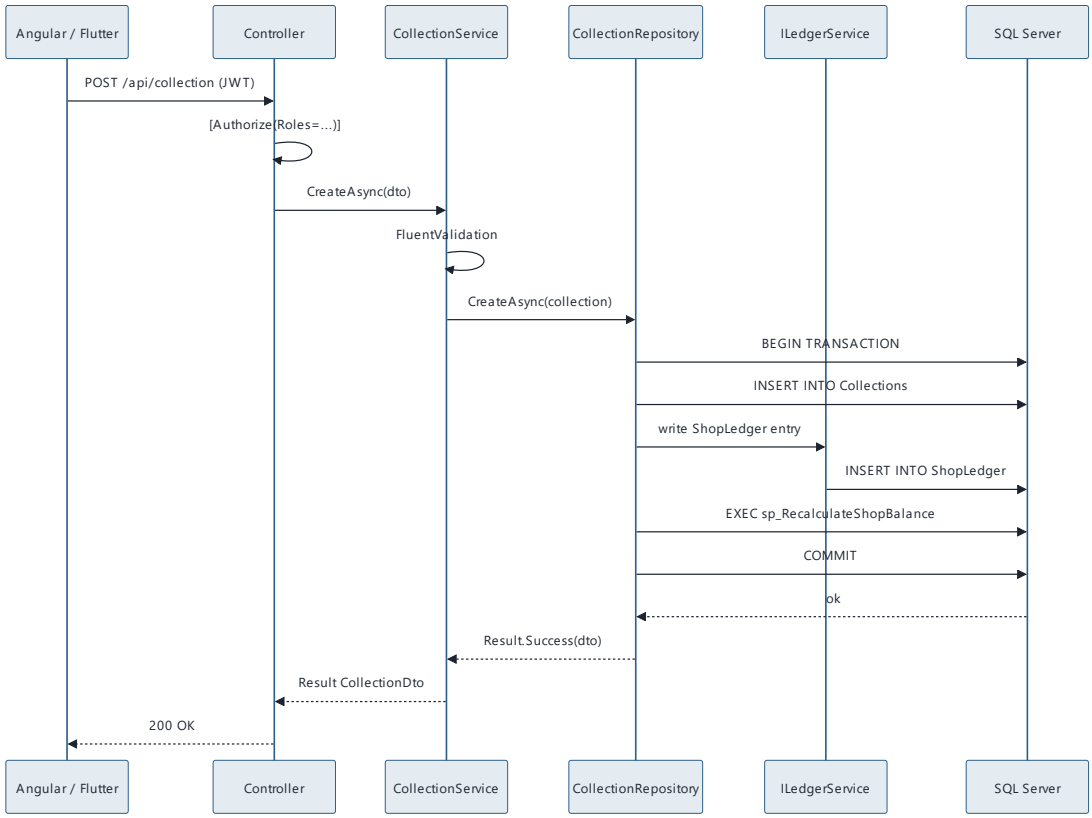
src/FruitWholesale.Api/Controllers/ApiControllerBase.cs

```
[ApiController]
[Authorize]
[Route("api/[controller]")]
public abstract class ApiControllerBase : ControllerBase
{
    protected ActionResult<T> FromResult<T>(Result<T> result) =>
        result.IsSuccess ? Ok(result.Value) : BadRequest(new { errors =
result.Errors });
}
```

A typical action is one line: `FromResult(await service.CreateAsync(dto))`. All the actual logic already happened in Application; the controller's only job is translating HTTP in and `Result<T>` out. `Program.cs` wires up JWT bearer auth, Swagger, Serilog request logging, CORS, and static file serving (for the uploaded company logo) once, at startup.

The pattern that holds the money together

This is the single most important convention in the backend. Every module that moves money or stock — Supply, Purchase, Collection, SupplierPayment, DailyExpense, EmployeeWorkLog — writes to *two* places at once: its own table, and a ledger (`ShopLedger`, `SupplierLedger`, `CashLedger`, or `StockLedger`). A single service, `ILedgerService`, is the only thing allowed to write to any ledger table — no repository writes ledger rows directly.



One SQL transaction wraps the insert, the ledger write, and the balance recalc — they succeed or fail together.

WHY IT MATTERS

A shop's *running balance* is never computed on the fly by adding up history — it's a stored, recalculated column, refreshed by a stored procedure inside the same transaction as the write that changed it. That's what makes the Dashboard's "Customer Outstanding" figure and a Shop Ledger's last-row balance always agree, even under concurrent writes from two different clients.

Authentication and the real access boundary

Login is a bcrypt password check against `Users.PasswordHash`, followed by a JWT with the user's role baked in as a claim. From then on, every request carries `Authorization: Bearer <token>`, and every controller (or individual action) can restrict itself with `[Authorize(Roles = "Admin,Manager")]`.

AREA	ADMIN	MANAGER	ACCOUNTANT	STAFF
Supply / Collections	✓	✓	✓	✓
Purchase, Supplier Payments, Expenses, Ledgers, Reports, Masters	✓	✓	✓	—
Settings (profile, logo, cash adjustment)	✓	—	✓	—
Users	✓	—	—	—

THE IMPORTANT BIT

This table is enforced *in the API*, with `[Authorize(Roles=…)]` on every controller. The Angular route guards and the Flutter drawer filtering below apply the identical rules — but only for a clean UI. If either client's UI check were removed entirely, the API would still reject the request. The clients are a convenience layer; the API is the lock.

Database: one schema script, then migrations

`database/01_CreateDatabase_Tables.sql` is the full schema — tables, stored procedures, a drop-and-recreate script for a clean environment. Every change made afterward, once real data existed, became its own numbered, idempotent script instead:

database/06_AddDiscountColumns.sql

```sql
IF NOT EXISTS (
    SELECT 1 FROM sys.columns
    WHERE object_id = OBJECT_ID('Collections') AND name = 'DiscountAmount'
)
BEGIN
    ALTER TABLE Collections ADD DiscountAmount DECIMAL(18,2) NOT NULL DEFAULT 0;
END
```

Running any migration script twice is a no-op — the guard checks whether the change already happened before touching anything, so scripts can be re-run safely without risking the live data.

---

02 Angular Web Client

Angular 20, standalone components throughout — no `NgModule` anywhere in the app — and signals for local state instead of the older `Observable`-heavy patterns.

Core: guards, services, models

`src/app/core/` holds everything cross-cutting:

- `guards/auth.guard.ts` — blocks the whole authenticated shell if there's no valid session.
- `guards/role.guard.ts` — `roleGuard('Admin','Accountant')` wraps individual routes, redirecting anyone else out.
- `services/auth.service.ts` — holds the logged-in user as a `signal`, exposes `hasRole(...roles)`.
- `services/branding.service.ts` — fetches the company name/logo once and shares it reactively, so the sidenav updates instantly after a Settings save with no page reload.
- `models/common.model.ts` — the same `USER_ROLES`, `PAYMENT_MODES` etc. constants that mirror the Domain enums on the backend.

One generic service class powers six CRUD modules

Shops, Suppliers, Fruits, Routes, Employees, and Expense Categories are all "list + activate/deactivate + create/edit" screens with slightly different fields. Rather than writing six near-identical Angular services, they all extend one generic base:

core/services/master-data-api.service.ts (shape)

```
export abstract class MasterDataApiService<TDto, TSaveDto> {
  getPaged(request): Observable<PaginatedList<TDto>>
  getAllActive(): Observable<TDto[]>
  create(dto: TSaveDto): Observable<TDto>
  update(id, dto: TSaveDto): Observable<TDto>
  activate(id) / deactivate(id)
}
```

A concrete service like `ShopMasterService` is then only a few lines — just the base URL and the type parameters. Every list component built on top of it gets pagination, search, and the active/inactive toggle for free.

The transaction modules (Supply, Purchase, Collection, SupplierPayment, DailyExpense, EmployeeWorkLog) follow a matching two-component pattern instead — a list component with a Material table and filters, and a form component with reactive forms whose validators mirror the backend's FluentValidation rules field-for-field, so the same "Amount must be greater than 0" rule shows up instantly client-side and is still re-checked server-side.

Material 3 theming

src/styles.scss

```
@include mat.theme((
  color: (
    theme-type: light,
    primary: mat.$blue-palette,
    tertiary: mat.$orange-palette,
  ),
  typography: Roboto,
));
```

Blue primary, orange tertiary — the same palette the Flutter app's `ColorScheme.fromSeed(seedColor: Colors.blue)` was chosen to match, so the two clients read as the same product. A global `MAT_FORM_FIELD_DEFAULT_OPTIONS` provider sets every

form field to `appearance: outline` with dynamic error-text sizing, fixing an early bug where long validation messages overlapped the next field.

Routing mirrors the role matrix exactly

src/app/app.routes.ts

```
{
  path: 'settings',
  loadComponent: () => import('./features/settings/settings.component'),
  canActivate: [roleGuard('Admin', 'Accountant')]
}
```

Every restricted route carries the exact same role list as its backend controller. The sidenav (`main-layout.component.ts`) filters its `NavGroup` / `NavItem` tree the same way, so a Manager never even sees a "Settings" link to click — and if they typed the URL directly, the route guard would bounce them to the dashboard anyway.

---

03 Android Client (Flutter)

Built after both the backend and the Angular app already existed, so it deliberately mirrors their structure rather than inventing a new one — same module boundaries, same role matrix, same visual language. State management is deliberately simple: `Provider` with `ChangeNotifier`, no Riverpod or Bloc, because the app's state is small — "who's logged in" is basically the only thing that needs to be observable app-wide.

Core layer

ApiClient
Wraps `http`, attaches the JWT, decodes JSON, throws `ApiException` on non-2xx. Caches the token in memory instead of re-reading secure storage on every call.

AuthService
`ChangeNotifier` holding the current user. Pushes the token into `ApiClient` on login/logout/restore.

TokenStorage
`flutter_secure_storage` — the session persists in the Android Keystore, not plain SharedPreferences.

AppTheme
Material 3, `ColorScheme.fromSeed(Colors.blue)` — deliberately matched to the Angular palette.

A small but real optimization lives in `ApiClient`: it used to call `await tokenStorage.read()` — a disk/IPC round-trip through the Android Keystore — before *every single HTTP request*. Now `AuthService` pushes the token into an in-memory field whenever it changes, and `ApiClient` just reads that field.

Feature modules — and the pagination fix

Each module (Supply, Purchase, Collections, Supplier Payments, Daily Expenses, Employee Salary, the six Masters, Ledgers, Users) is a model file, a service file, a list screen, and a form screen — deliberately parallel to the Angular module shape. Two shared widgets do most of the heavy lifting:

lib/core/widgets/paginated_list_view.dart

```dart
class PaginatedListView<T> extends StatefulWidget {
  final Future<PaginatedList<T>> Function(int page) fetchPage;
  final Widget Function(BuildContext, T) itemBuilder;
  // handles: initial load, pull-to-refresh, and
  // infinite scroll — loads page N+1 automatically
  // once the user scrolls near the bottom
}
```

> **A REAL BUG THIS FIXED**
>
> Early versions of every list screen called `getPaged()` once, with no way to load page 2 — meaning any list past ~20 records silently hid the rest. `PaginatedListView` replaced that everywhere at once: 17 screens, one fix, plus proper infinite scroll instead of a "next page" button.

`MasterListScreen<T>` builds on top of it for the six Masters modules specifically, adding the active/inactive `Switch` and the "add new" FAB — the same role `MasterDataApiService<T>` plays on the Angular side, just for the list UI instead of the HTTP calls. Forms share `ErrorBanner`, `SaveButton` / `SaveFooterBar`, and `DatePickerField` — the same red-banner, spinner-button, and date-picker code that used to be copy-pasted into all twelve form screens.

Navigation: the drawer as a role filter

`HomeShell` builds its drawer from a list of `NavGroup`s, each holding `NavItem`s with an optional `roles:` list — structurally identical to Angular's sidenav config, right down to the group names (Overview, Transactions, Ledgers, Reports, Masters, Administration):

lib/features/home/home_shell.dart

```
NavItem(
  label: 'Settings',
  builder: (_) => const SettingsScreen(),
  roles: [UserRole.admin, UserRole.accountant],
)
```

A Staff login sees exactly two drawer items — Supply and Collections — because every other `NavItem` carries a `roles:` list that excludes them, and their "home" screen on login is Supply rather than the Dashboard they're not allowed to see.

Release build: signed, shrunk, real

`android/app/build.gradle.kts` reads a git-ignored `key.properties` file to sign release builds with a real keystore (falling back to the debug key only when that file is absent, so a fresh clone still builds). R8 handles minification, resource shrinking, and obfuscation.

COMMAND	OUTPUT
`flutter build apk --release`	Universal APK, ~52 MB
`flutter build apk --release --split-per-abi`	Per-architecture APKs, 16–20 MB each
`flutter build appbundle --release`	.aab for Play Store, ~51 MB

The one idea that ties all three together

Neither client is trusted with anything important. Validation rules, ledger consistency, and the Admin/Manager/Accountant/Staff matrix all live exactly once, in the API. Angular and Flutter are both, structurally, the same shape repeated twice: a thin service per module calling a handful of REST endpoints, a list screen, a form screen, and a role-filtered nav item pointing at both. Learn one client's module pattern and you can find your way around the other by pattern-matching alone — which is exactly why the Flutter app could be built to full feature parity by mirroring decisions the Angular app had already made.

Fruit Wholesale Management System — architecture reference. Backend: ASP.NET Core 10 / Dapper / SQL Server. Web: Angular 20. Mobile: Flutter (Android).